

Musanze HarvestLink Cooperative

Harvest & Dispatch Decision Pipeline

SWE 3513 - Artificial Intelligence

Assignment 1 - Group AI-G01

Group Members:

[Member 1] - Data & UX Lead

[Member 2] - Regression Engineer

[Member 3] - Classification Engineer

[Member 4] - Clustering & QA Engineer

[Member 5] - Reproducibility & Release Lead

Dataset: AI_A1_G01.csv (30 farms, 6 features)

SHA-256: 70480135477a038edb41ec136b249f606176e616acf8c64a523773133f52c762

1. Project Overview

Purpose:

Provide Musanze HarvestLink Cooperative with three operational decisions before potato dispatch:

- (1) Estimate expected harvest weight per collection point
- (2) Flag consignments needing priority dispatch attention
- (3) Group collection points with similar operating profiles

Input Data: data/AI_A1_G01.csv (30 records, group-specific)

record_id	text	Unique identifier (not a feature)
plot_area_ha	numeric	Farm area (hectares)
rainfall_mm	numeric	Recent rainfall estimate
soil_ph	numeric	Soil acidity measure
seed_kg	numeric	Seed quantity (kg)
distance_km	numeric	Distance to collection point
arrival_hour	numeric	Planned arrival hour (24h)
actual_yield_kg	target	Regression target
dispatch_attention	target	Classification target (0/1)

Three Decisions & Models:

1. Harvest Weight	Regression	Linear Reg. (NumPy GD)	MAE=51, R2=0.995	Yield (kg)
2. Dispatch Flag	Classification	Logistic Regression	Acc=1.0, F1=1.0	0/1 + Prob
3. Farm Groups	Clustering	K-Means (k=3)	Silhouette=0.58	Cluster 0/1/2

2. Technical Approach

Data Pipeline (src/data_pipeline.py)

- Load CSV, validate column names & types against schema
- Report missing values, duplicates, descriptive statistics
- Compute SHA-256 fingerprint for audit trail
- Separate identifiers from features; produce NumPy feature matrix
- Save: data_report.json (row count, feature count, stats, SHA-256)

Regression from First Principles (src/regression.py)

- Linear regression with batch gradient descent using only NumPy
- Feature standardization (fit on train, transform test)
- Train/test split (80/20, seed=42)
- Loss history tracked per epoch; early stopping on tolerance
- Metrics: MAE, RMSE, R-squared
- No sklearn estimator used for this section
- Save: regression_metrics.json, regression_loss.png, regression_model.npz

Classification (src/classification.py)

- Logistic Regression with sklearn (pandas + sklearn permitted)
- Stratified train/test split (seed=42)
- StandardScaler fit on training data only
- Metrics: Accuracy, Precision, Recall, F1, Confusion Matrix
- Cost analysis: False Negative (missed dispatch) > False Positive
- Save: classification_metrics.json, confusion_matrix.png, classification_model.joblib

Clustering (src/clustering.py)

- Standardize features (StandardScaler)
- Evaluate k = 2, 3, 4, 5 using silhouette score
- Select k=3 (silhouette=0.58)
- KMeans with n_init=10, random_state=42
- PCA 2D projection for visualization
- Cautious interpretation: statistical groups, not verified categories
- Save: clustering_metrics.json, clusters.csv, cluster_plot.png, clustering_model.npz

3. Commands & Generated Artifacts

Run Complete Pipeline:

```
python run_all.py --data data/AI_A1_G01.csv --output artifacts/ --group AI-G01
```

Run Prediction (Python one-liner):

```
python -c "  
import sys; sys.path.insert(0, ".")  
from src.regression import load_regression_model, predict_regression  
from predict import load_classification_model, load_clustering_model,  
predict_classification, predict_clustering  
from pathlib import Path; import numpy as np  
X = np.array([[1.2, 81, 5.7, 210, 14, 9]], dtype=float)  
w,m,s = load_regression_model(Path("models/regression_model.npz"))  
print("Yield:", predict_regression(X,w,m,s)[0])  
c = load_classification_model(Path("models"))  
print("Dispatch:", predict_classification(X, c["model"], c["scaler"])[0][0])  
cl = load_clustering_model(Path("models"))  
print("Cluster:", predict_clustering(X, cl["cluster_centers"],  
cl["scaler_mean"], cl["scaler_std"])[0])  
"
```

Artifacts in artifacts/:

data_report.json	Schema validation, stats, SHA-256 fingerprint
regression_metrics.json	Weights, scaler, MAE/RMSE/R2, hyperparameters
regression_loss.png	Training & validation loss curves (MSE)
classification_metrics.json	Acc/Prec/Rec/F1, confusion matrix
confusion_matrix.png	Heatmap of predictions vs actual
clustering_metrics.json	Silhouette scores k=2..5, best k, inertia
clusters.csv	record_id, cluster_label for all 30 farms
cluster_plot.png	PCA 2D projection colored by cluster

Models in models/:

regression_model.npz	Weights (7), scaler_mean (6), scaler_std (6)
classification_model.joblib	LogisticRegression + StandardScaler
clustering_model.npz	Cluster centers (3x6), scaler params, k=3

4. Validation & Constraints

Prediction Input Validation (predict.py):

- All 6 required fields: plot_area_ha, rainfall_mm, soil_ph, seed_kg, distance_km, arrival_hour
- All values must be numeric (int or float)
- Returns clear JSON errors for missing fields, wrong types, invalid JSON, missing models

Assignment Constraints Satisfied:

- Regression from scratch: NumPy batch gradient descent; no sklearn estimator
- Fixed random seed: SEED=42 recorded in all metrics files
- No data leakage: Scalers fit only on training data
- No hardcoded outputs: Works on hidden test data with same schema
- Stratified split: Classification uses stratified train/test
- Silhouette-based k: k=2..5 evaluated; best k=3 selected
- Cautious clustering: Clusters are statistical, not verified categories
- Clean environment: Runs from fresh venv with requirements.txt
- SHA-256 fingerprint: Computed at runtime, saved in data_report.json
- Reproducible artifacts: All JSON/CSV/PNG generated by code

5. Results Summary

Regression Metrics:

MAE	50.96 kg
RMSE	59.15 kg
R-squared	0.9949
Learning Rate	0.01
Max Epochs	2000 (early stop at 1e-6)
Final Train Loss	5509.21
Final Val Loss	3495.96

Classification Metrics:

Accuracy	1.0000
Precision	1.0000
Recall	1.0000
F1 Score	1.0000
Test Size	0.2 (6 samples)
Train Dist.	16 class 0, 8 class 1
Test Dist.	4 class 0, 2 class 1

Clustering Results:

Best k	3
Silhouette	0.5801
Inertia	156.7
k=2 Silhouette	0.3892
k=4 Silhouette	0.3987
k=5 Silhouette	0.3654

Example Prediction:

Input: plot_area_ha=1.2, rainfall_mm=81, soil_ph=5.7, seed_kg=210, distance_km=14, arrival_hour=9

Predicted Yield	2684.4 kg
Dispatch Flag	0 (No attention needed)
Dispatch Probability	0.46
Cluster Assignment	1

6. Submission Package

Files to Submit (Group Leader Only):

- AI_A1_G01.zip - Full project (excludes .env, __pycache__, .ipynb_checkpoints)
- AI_A1_G01_UIUX.pdf - 5-7 page dashboard design document
- AI_A1_G01_CONTRIBUTIONS.pdf - Signed contribution statements

Moodle Online Text Required:

- Group number: AI-G01
- Group verification code: [provided by lecturer]
- Group leader: [name]
- Group members: [5 names]
- GitHub repository URL: [URL]
- Final commit hash: [git hash]
- Dataset SHA-256: 70480135477a038edb41ec136b249f606176e616acf8c64a523773133f52c762
- Demonstration video link: [if not in ZIP]

Demonstration Video (90s, continuous, unedited):

- Terminal clock visible throughout
- Group code, Git commit hash, dataset SHA-256 shown
- Run full pipeline: python run_all.py ...
- Show generated JSON, CSV, PNG, model files
- Open one metrics JSON and one plot
- Explain one model result in own words
- Run predict.py with valid input
- Run predict.py with invalid input (show validation error)
- Show UI/UX PDF and Contributions PDF filenames
- Student states name, registration number, assigned role